

Developer Integration Guide

Generate student ID cards from your app with a handful of clean calls. This guide shows the rendered card layout, then covers the Python SDK, the REST web API, and the web server for template previews.

Python · FastAPI · Open Source

Implementing the SDK — main.py

This is the working entry point the developer implemented. It picks an installed template, fills a `Student` with a live Kaascan record, adds branding assets (stamp, signature, barcode) and hands it to the async card agent. We updated `main.py` so images render and templates load correctly.

```
main.py

from card_sdk.card_agent import Card, Student
from card_sdk.template_manager import pull_template_files, pull_template_options
from card_sdk.card import image_file_to_base64
from card_sdk.base64qrcode import base64_barcode, base64_qrcode

import asyncio

# pick the installed SDK template to generate with (menu choice in the UI)
template = pull_template_options()[0] # e.g. 'BLUE GREEN STUDENT CARD'

# --- configure the card data -----
Student.photo = "https://admin.kaascan.com/assets/2ebf1446-...?download="
Student.name = "Tuyishimire Fraterine Ely"
Student.Class = "Software Development"
Student.school_name = "KANYINYA TSS 001"
Student.student_id = "cvvvvvvv"
Student.template_name = template
Student.data_qrcode = "{}" # empty dict = no QR (this card uses a barcode)

# --- stamp, signature & barcode (rendered on the card) -----
Student.stamp = image_file_to_base64("draf/root/stamp.png")
Student.signature = image_file_to_base64("draf/root/signature.png")
Student.barcode = base64_barcode({"code": Student.student_id})

student = Student(
    photo=Student.photo, name=Student.name, Class=Student.Class,
    school_name=Student.school_name, data_qrcode=Student.data_qrcode,
```

```
student_id=Student.student_id, template_name=Student.template_name,  
stamp=Student.stamp, signature=Student.signature, barcode=Student.barcode,  
)  
  
asyncio.run(Card.agent(data=student))
```

What we updated

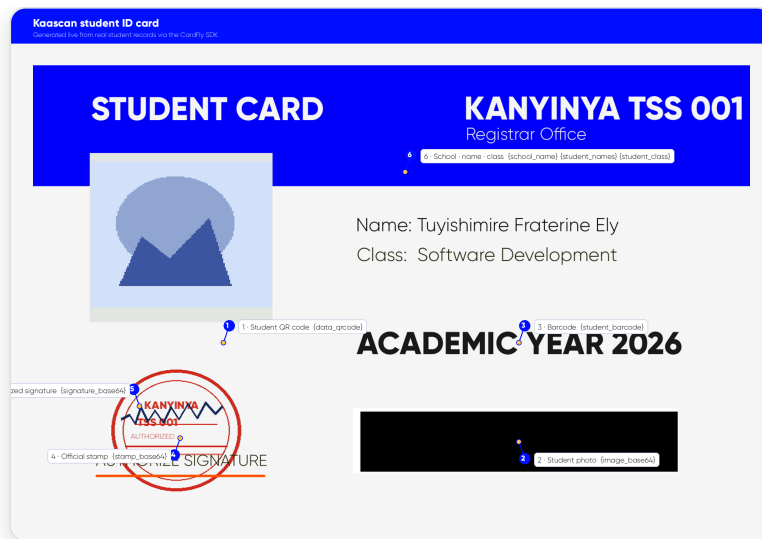
`image_file_to_base64()` returns a bare base64 string; the renderer now promotes it to a proper `data:` URI so the stamp & signature are visible, and the barcode is generated as a data-URI PNG so it renders as-is. Templates are discovered via `pull_template_options()` so `main.py` always targets a real installed template.

Rendered Card Layout

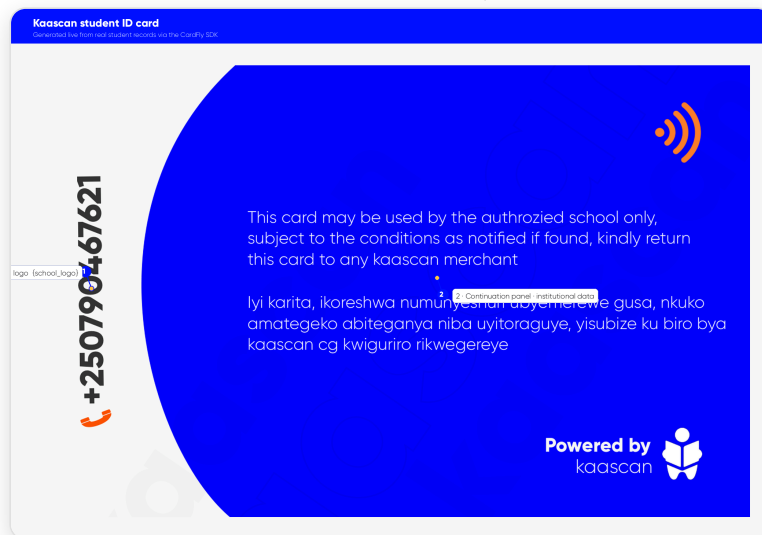
Every `{placeholder}` in a template is a live field, fed by a real student record. On this card the fields map straight onto the columns of the Kaascan students API.



Live-produced preview — front & reverse of the generated card from a real student record.



Front face field map.



Reverse face field map.

Card region	Card parameter	Kaascan field / typical value
Student photo	<code>image_base64</code>	<code>student.photo</code> (PNG, proxy-served)
Student name / class	<code>student_name</code> , <code>student_class</code>	<code>student.name</code> , <code>student.class</code>
School name	<code>school_name</code>	<code>student.school_name</code>
Barcode	<code>student_barcode</code>	base64 PNG
Official stamp	<code>stamp_base64</code>	base64 PNG
Signature	<code>signature_base64</code>	base64 PNG
QR payload	<code>data_qrcode</code>	<code>https://kaascan.com/verify/<student_id></code> — only if the card uses a QR
School logo	<code>school_logo</code>	base64 PNG (reverse face)

Barcode or QR — not both

When the template renders a **barcode**, leave `data_qrcode` as an empty dict (`data_qrcode = {}`). The worker treats an empty QR payload as “no QR”, so it suppresses the QR image entirely and does not log a spurious “missing QR” warning. Supply a real QR payload only for cards that display a QR code instead of (or in addition to) a barcode.

Live preview by school

The CardFly editor can pull a real student through the API, fill its `{placeholders}` , and re-render the card in preview mode instantly — so the layout above is generated from actual records, not mock data.

Quick Start

Install the dependencies and start the preview server in two commands.

```
Terminal

pip install -r requirements.txt
# start the web UI (optional, for template previews)
python -m card_sdk.web_server
```

Note

The package lives in `sdk/card_sdk/` . Add the `sdk/` folder to your `PYTHONPATH` or install it so `import card_sdk` resolves to the repo copy.

Generating a Card

For a single card, `generate_card()` is all you need. It returns a status string and writes the finished PDF to the output folder.

```
example.py

from card_sdk.card import generate_card, image_file_to_base64

result = generate_card(
    template_name="BLUE GREEN STUDENT CARD",
    image_base64=image_file_to_base64("student.jpg"),
    student_name="Tuyishimire Fraterine Ely",
    student_class="Software Development",
    school_name="KANYINYA TSS 001",
    student_id="S-2026-001",
    data_qrcode="https://kaascan.com/verify/S-2026-001",
```

```
)
print(result) # status message + output path
```

Output goes to `output/<school_name>/<class>/final_student_card_<id>.pdf`.

Choose a template

Python

```
from card_sdk.template_manager import pull_template_options, pull_template_files

options = pull_template_options() # e.g. ["BLUE GREEN STUDENT CARD", "RUNO SCHOOL CARD", ...]
```



One call

`generate_card()` renders the card and writes the final PDF automatically.



Template list

`pull_template_options()` returns every installed template by name.



Extensible

Branding assets, colors and typography are all optional keyword arguments.

Config Object

Wrap your data in a `CardConfig`. It centralises the values and can build a pydantic `Student` model for the asynchronous batch flow.

config_usage.py

```
from card_sdk.card import CardConfig, image_file_to_base64

photo = image_file_to_base64("student.jpg")

cfg = CardConfig(
    photo=photo, # required: base64 student photo
    template_name="BLUE GREEN STUDENT CARD",
    student_name="Tuyishimire Fraterine Ely",
    student_class="Software Development",
    school_name="KANYINYA TSS 001",
    student_id="S-2026-001",
    data_qrcode="https://kaascan.com/verify/S-2026-001",
    # optional branding: school_logo, stamp, signature, barcode (all base64)
)

# Option A: render the card content dict
content = cfg.generate_card_content() # renders card + writes PDF
```

```
# Option B: build a Student model for the async batch API
student = cfg.to_student() # card_sdk.card_agent.Student instance
```

Branding Assets

Pass base64-encoded assets as optional keyword arguments. Every parameter has a sensible default, so integrations only need to send what they have.

```
branding.py

from card_sdk.card import generate_card, image_file_to_base64

result = generate_card(
    template_name="BLUE GREEN STUDENT CARD",
    image_base64=image_file_to_base64("student.jpg"),
    student_name="Alice Johnson",
    student_class="Grade 4",
    school_name="SUNRISE ACADEMY",
    student_id="A-001",
    data_qrcode="https://kaascan.com/verify/A-001",
    # --- optional branding ---
    school_logo=image_file_to_base64("logo.png"),
    stamp_base64=image_file_to_base64("stamp.png"),
    student_barcode=image_file_to_base64("barcode.png"),
    signature_base64=image_file_to_base64("signature.png"),
    side_2_color="#00897B",
)
```

Image helpers

Helper	Description
<code>image_file_to_base64(path)</code>	Read a local image file and base64-encode it.
<code>image_url_to_base64(url)</code>	Synchronously fetch a remote image and encode it.
<code>resize_image_base64(b64, w=100, h=100)</code>	Resize an encoded image for smaller cards.

Web Server

Start the FastAPI web UI from the terminal. This is what the terminal UI's "Start the web server to view templates?" prompt calls for you.

```
Terminal
```

```
python -m card_sdk.web_server # port 8000
python -m card_sdk.web_server --port 9000
python -m card_sdk.web_server --host 127.0.0.1
```

Or start it from code:

```
from card_sdk.web_server import serve_web
serve_web(host="0.0.0.0", port=8000) # blocks until interrupted
```

The server binds to the local dev SDK automatically (it adds the repo to the path), so you always see the real templates.

REST API

Once the server is running, call the endpoints from any client (curl, JavaScript, Postman, ...).

Method	Endpoint	Purpose
GET	/healthy/status	Health check. → {status, time_stamp}
GET	/card/templates	List template names + preview paths. → [{template_name, front_preview_path, back_preview_path}]
GET	/student/photo/<student_id>	Get a cached student photo (PNG).
GET	/student/card/<front back>/<student_id>	Get a rendered card side as SVG.

```
JavaScript / fetch

const res = await fetch("http://localhost:8000/card/templates");
const templates = await res.json();
console.log(templates.map(t => t.template_name));
```

```
curl
curl http://localhost:8000/card/templates
curl http://localhost:8000/student/card/front/S-2026-001
```

Tip

Interactive API docs are available at <http://localhost:8000/docs> (Swagger UI).

Reference

generate_card()

```
Signature

generate_card(
    template_name: str,
    image_base64: str,          # student photo (base64)
    student_name: str,
    student_class: str,
    school_name: str,
    student_id: str,
    data_qrcode: str,          # payload encoded into the QR code
    side_2_color="#00897B",
    school_logo="",
    stamp_base64="",
    student_barcode="",
    signature_base64="",
    snFontSize=2.4932, snx=-10.523738, sny=-0.8769781,
) -> str
```

Public entry points

Symbol	From	Notes
<code>generate_card</code>	<code>card_sdk.card</code>	Render a single card (returns status string).
<code>save_card(content, id, class, school)</code>	<code>card_sdk.card</code>	Convert card SVG content to the final PDF.
<code>CardConfig</code>	<code>card_sdk.card</code>	Config object → <code>generate_card_content()</code> / <code>to_student()</code> .
<code>Card</code>	<code>card_sdk.card_agent</code>	Async batch entry point — <code>await Card.agent(data=student)</code> .
<code>Student</code>	<code>card_sdk.card_agent</code>	Pydantic model (photo, name, Class, school_name, data_qrcode, student_id, template_name).
<code>pull_template_options</code>	<code>card_sdk.template_manager</code>	List of template names.
<code>pull_template_files</code>	<code>card_sdk.template_manager</code>	Detailed template file list (front/back paths).
<code>serve_web</code>	<code>card_sdk.web_server</code>	Start the FastAPI web UI.

Tip

The effective card *content* (SVG) lives in the returned value of `generate_card()` & `CardConfig.generate_card_content()` ; the final PDF is produced by `save_card()` / the card agent. For most integrations, calling `generate_card()` is enough — the PDF is written automatically.